
ReconX — Enterprise Trade Reconciliation Platform

Deutsche Bank TDI 2026 training case study

Evidence boundary first. Demonstration detail second.

TRAINING CASE STUDY · NOT PRODUCTION DEPLOYMENT

Mismatches need a controlled path

One trade, two records, one break to investigate.

ReconX models the operator's journey from capture to comparison to evidence-backed resolution.

- Capture the internal trade record.
- Compare it with an external record.
- Investigate and resolve the break with evidence.

ILLUSTRATIVE DISCREPANCY · NOT PRODUCTION DATA

	INTERNAL RECORD	EXTERNAL RECORD
trade reference	same reference	same reference
quantity	captured	received
price	recorded value	received value

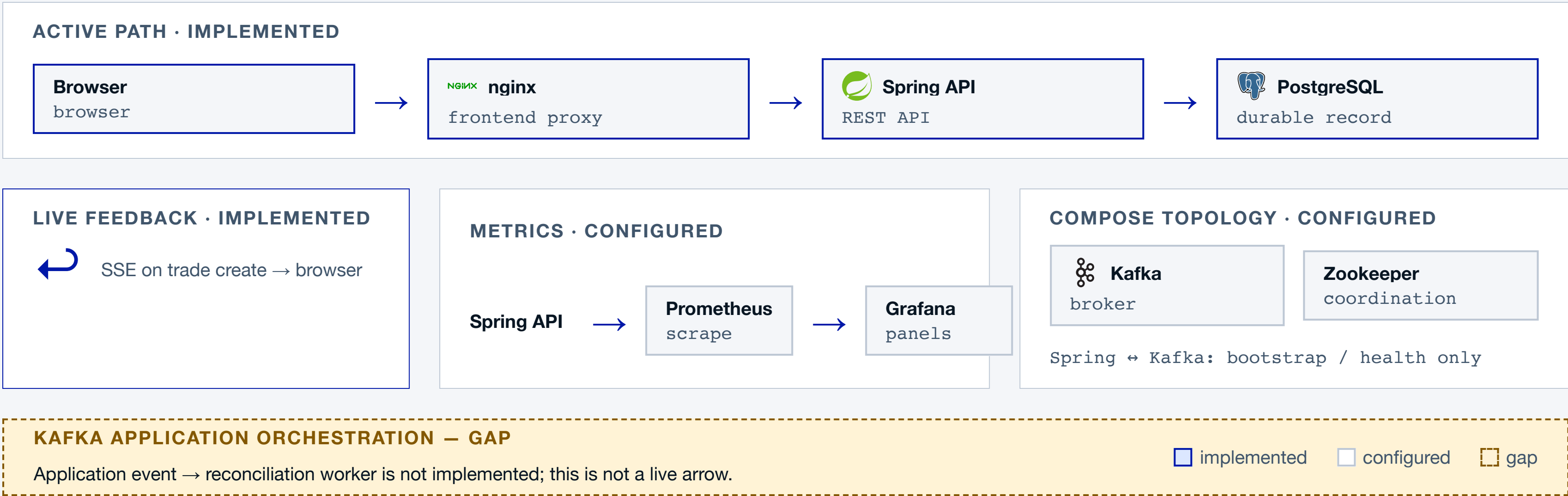
BREAK

A mismatch is now inspectable: compare the records, then resolve with evidence.

Architecture, as it runs today

SOURCE-SUPPORTED PATH

Capture and observation are source-supported; event orchestration remains unfinished.



Stack by responsibility, not by shopping list

A FOUNDATION WITH EXPLICIT MATURITY BOUNDARIES

01	DOMAIN / API	IMPLEMENTED	 Java 25	 Spring Boot 3.5.0	sealed types · DTO boundaries	
02	DATA / AUDIT	IMPLEMENTED	 PostgreSQL 16	Liquibase · JPA / Envers		JSONB · partitions
03	UI / LIVE UPDATES	IMPLEMENTED	 React 19	 Vite 6	 nginx	SSE
04	MESSAGING / OPS	CONFIGURED	 Kafka scaffold	 Prometheus 2.54.1	 Grafana 11.2.0	application path unfinished
05	DELIVERY / PRESENTATION	CONFIGURED	 Docker	Compose	 GitHub Actions verification	Reveal.js 6 · Playwright

Messaging is a configured dependency, not yet an integrated application path. Release status requires separate evidence; version strings alone are not release proof.

Live demo runway

Login → validated trade → live update. The slide stays useful if the app does not.



RUNTIME CAPTURE PENDING Launch the guarded frame when ready / If unavailable: full-demo link or source/API walkthrough

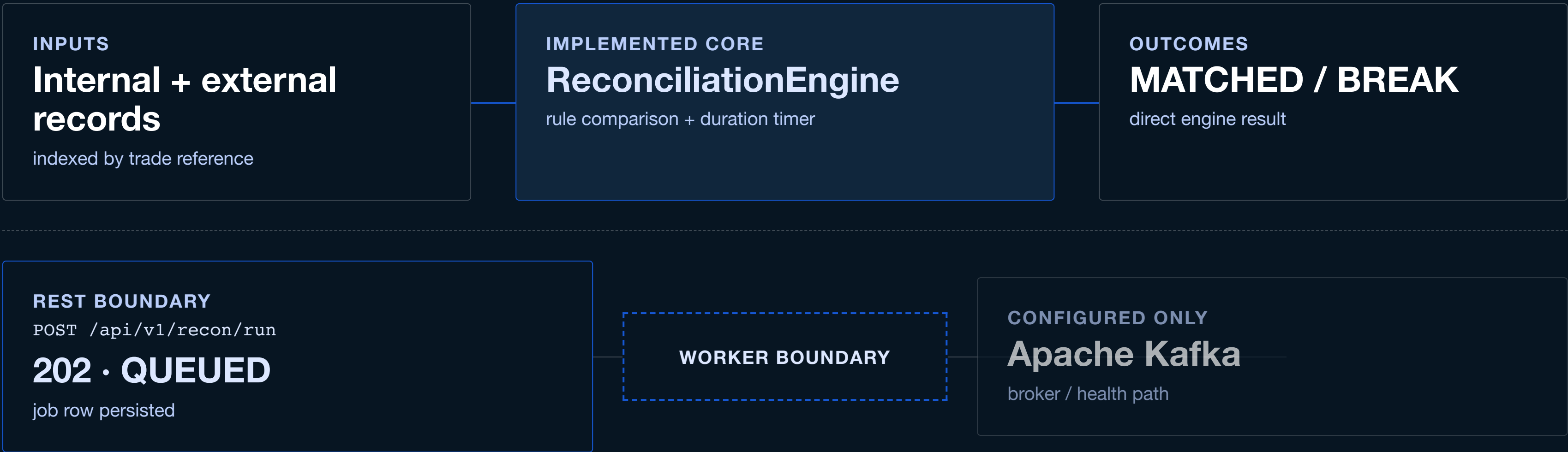
Demo: idle

Launch embedded demo

Open full demo

Reconciliation truth line

Core logic is real. The seam is visible.



Not observed

Queue has no worker; no application listener result is observed.

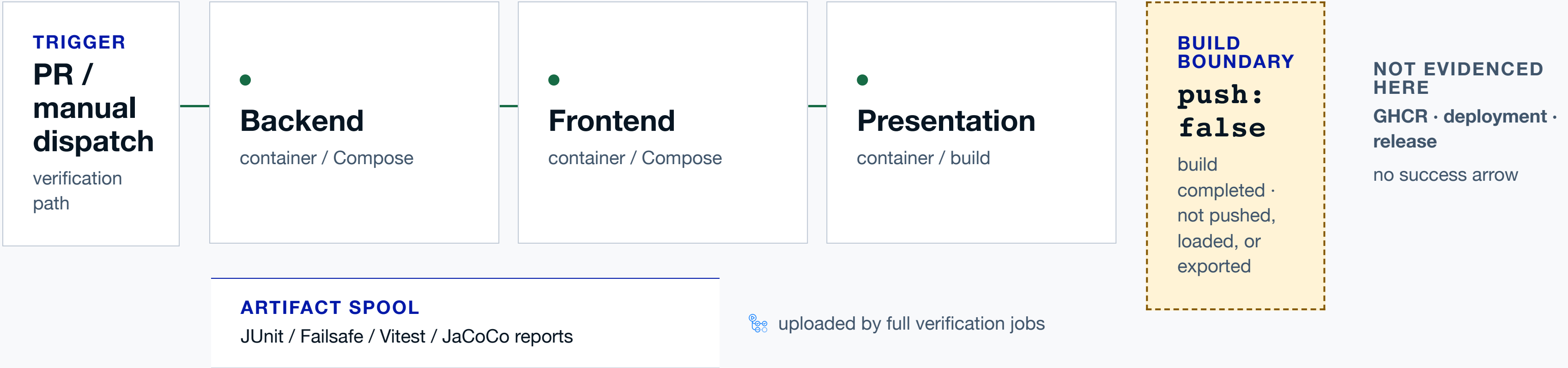
CI verifies the build — then stops

A commit-bound verification rail, not a release claim.

VERIFIED REMOTE RUN

PR #272 · run 30898100384

commit 4f1a939a



Passed on the named run

Backend, frontend, and presentation verification passed; JUnit/Vitest report checks passed.
Image builds completed without push, load, or export. Load job and native fallbacks: skipped.

Monitoring is provisioned; load proof is pending

The source has a dashboard. The run still needs to be captured.

EVIDENCE PATH
source → runtime → recovery

01

Five panels are defined.

- + request rate
- + P95 latency
- + open breaks
- + trades / second
- + reconciliation duration

reconx-backend scrapes
/api/actuator/prometheus.

02

No current image.

runtime capture pending

target state + panel values require a fresh run

A dashboard definition is not a measured screenshot.

03

The profile is on disk.

10 VUs × 10 iterations =
100 trade creations

Zero-failure checks are configured; no completed run is currently proven.

Next capture
200-VU baseline / load / recovery

Configured now · fresh runtime evidence required

Keep the current 10-VU / 100-create truth. Do not promote a 200-VU success, latency SLA, or recovery claim without a captured run and provenance.

Learnings we can defend

1:35 • reflection before the close

01

Configuration is not integration.

Promote a path only after its behavior is observable end to end.

02

Contract drift breaks rehearsals.

Keep the current request shape beside the walkthrough.

03

Evidence needs provenance.

Attach source, run, and capture context to every claim.

DELIVERY CHECK

Presenter reflection pending

Personalize with confirmed sentences before delivery. Keep the prompts team-level until then.

No attribution • no rehearsal outcome • no invented voice

Questions?

Follow the evidence to the layer that needs a closer look.

ROUTE THE QUESTION

API

Trade boundary and request contract

Recon

Matching core and orchestration seam

Runtime

Compose path and observability

Delivery

Verification and source provenance

REPOSITORY / SOURCE

<https://github.com/anushadagar1407/db2026-reconX>

OPEN ANCHORS TradeController ReconciliationEngine docker-compose.yml

A sourced answer is better than a finished-sounding one.